

DA3408 Assignment 2

Rinkesh Patel DA24B017

Q1 Single-Stage vs. Multi-Stage Docker

Naive build

So in the naive Dockerfile build it used a single `FROM python:3.12` stage where it essentially copied the whole build context which runs a command `pip install -r requirements.txt` and then `python train.py` in place and in last the uvicorn server is served for the API. It builds and runs correctly: checking `/healthz` returns `{"status":"ok"}` and for `POST /predict` on a spam message it correctly gave us the label `{"label":"spam"}`. The size of the image was **1.59 GB** as shown in evidence.

Multi-stage build

Then for the multi-stage build we kept the same `python:3.12` builder stage to install dependencies and run `train.py` but the final runtime stage also uses `python:3.12`, so nothing switches to a slim base here. Here we only copied over what was actually needed to run which includes `site-packages` binaries the saved `model.joblib` and the `app/` code. We tested both `/healthz` and `/predict` endpoints again and got the exact same correct results. The resulting image size came out to **1426.7 MiB (approx. 1.5 GB)** which is only a **5.72% reduction** compared to the naive build.

Why it's smaller

Because the final stage is also `python:3.12` rather than a slim variant, it already contains its own copy of the Debian build tools, gcc, header files, and apt lists inherently, regardless of the multi-stage split, so multi-stage buys us nothing on that front here. The entire remaining size difference (about 86.6 MiB) comes down to a narrower cause: pip's download cache never gets copied over, since the builder stage installs with `--no-cache-dir` and the `COPY --from=builder` pattern only pulls specific paths into the final image, and in the naive build that pip cache measured around 91 MB on its own. On top of that, `requirements.txt`, `train.py`, and the 1,000-row `spam_dataset.csv` stay stuck in the discarded builder stage, so the final image only carries the pre-trained model and runtime dependencies rather than the training inputs.

Q2 Multi-Container Orchestration with Docker Compose

Cache-aware API

In `app/main.py` we set up a Redis client using environment variables `REDIS_HOST` and `REDIS_PORT`. When a request hits `POST /predict` it first checks Redis using the key `predict:{text}`. If there is a cache hit it returns the cached label right away without calling the model. If it is a miss it runs model prediction stores the label in Redis with a TTL of 300 seconds using `cache.setex` and returns the label. Both cache hit and miss paths were tested and returned correct predictions.

docker-compose.yml

I wrote a `docker-compose.yml` file that defines two services: `api` built from our multi-stage Dockerfile and `cache` using the official `redis:7-alpine` image. Docker Compose automatically connects them on a shared bridge network so the API container can reach Redis directly using the service name `cache` as its hostname with `REDIS_HOST=cache`.

Cache-hit evidence

When we brought up the stack with `docker compose up --build` and sent the same request twice the logs showed a clear speedup:

```
api-1 | CACHE MISS key='predict:a brand new message never sent before' took=29.45ms
api-1 | INFO: 172.22.0.1:33334 - "POST /predict HTTP/1.1" 200 OK
api-1 | CACHE HIT key='predict:a brand new message never sent before' took=0.35ms
api-1 | INFO: 172.22.0.1:33348 - "POST /predict HTTP/1.1" 200 OK
```

The first request was a cache miss taking 29.45 ms while the second request was a cache hit taking just 0.35 ms which is about **84× faster** because it bypassed the TF-IDF feature extraction and model inference completely.

Compose vs. Kubernetes

Docker Compose helps manage multi-container applications on a single host by defining networking builds and containers together in one declarative file instead of running multiple manual `docker run` commands. However Compose only works on a single machine and does not handle cluster scheduling automatic pod scaling or zero-downtime rolling updates. That is where Kubernetes comes in as it handles container orchestration across multi-node clusters.

Q3 Kubernetes Indexed Job: Parallel Data Validation

Shards and manifest

Using `generate_shards.py` we deterministically created 8 CSV shard files of 50 rows each with deliberately introduced invalid emails and missing fields. The resulting invalid row counts per shard were 10, 15, 10, 8, 6, 9, 13 and 11 for shards 0 to 7. Given a cluster with 2 nodes and 2 CPUs each making 4 total CPUs and each pod requesting 500m CPU with limit 500m running with `parallelism: 4` ensures that at most 4 pods run concurrently using 2 CPUs total or 1 CPU per

node. I picked `parallelism: 4` instead of 8 because running 8 pods at 500m would consume 100% of allocatable CPUs simultaneously leaving no CPU headroom for system components like kubelet CoreDNS and kube-proxy. This choice keeps 50% CPU capacity free per node to prevent pods from getting stuck in `Pending` state due to resource starvation. The manifest uses `completions: 8` `completionMode: Indexed` `restartPolicy: Never` and uses Downward API env vars `POD_NAME` and `NODE_NAME` so each pod knows where it ran.

Parallelism evidence

Running `kubectl get pods -o wide` while the job was executing showed pods 0–3 in `Completed` status after 7 seconds while pod 4 was still in `Pending`. A second snapshot a few seconds later showed pods 4–7 running after the first 4 finished. This proves that the job actually ran with a concurrency of 4 pods at a time rather than launching all 8 at once.

Results collection

I retrieved the invalid row counts for each shard using the Kubernetes API with `kubectl logs -l batch.kubernetes.io/job-name=--prefix=true` instead of using a shared volume. We avoided using a shared volume because minikube's default storage provisioner creates single-node hostPath PersistentVolumes. Since our pods were scheduled across multiple nodes like `minikube` and `minikube-m02` they would not be able to access a single hostPath volume concurrently across nodes without additional NFS setup. Reading pod logs via `kubectl logs` works reliably across nodes.

6-CPU re-justification

If the cluster had 3 nodes with 2 CPUs each making 6 total CPUs we would change `parallelism` to **6**. Requesting 500m CPU per pod across 6 pods uses 3 CPUs total or 1 CPU per node which maintains the same 50% headroom margin on each node for system processes while allowing the 8-completion job to finish faster in two waves of 6 and 2.

Q4 Kubernetes Deployments: Self-Healing & Rolling Updates

Manifests

We created `deployment.yaml` with `replicas: 2` CPU requests and limits set to 250m and a readiness probe querying `GET /healthz:8000`. We also created `service.yaml` to expose it via a ClusterIP Service on port 80. Applying both manifests brought up 2 ready replicas successfully.

Self-healing

To test self-healing we manually deleted one pod using `kubectl delete pod spam-api-77f4dcbc9b-5251q`. Checking `kubectl get pods` right after showed that Kubernetes automatically spun up a replacement pod `spam-api-77f4dcbc9b-k57jd` which was already `Running`. This happens because the Deployment's **ReplicaSet controller** continuously compares the actual number of running pods against the desired replica count of 2. As soon as it noticed the deleted pod was gone it triggered a reconcile loop to create a new pod from the pod template.

Rolling update

We updated `app/main.py` so `/healthz` returns `{"status":"ok","version":"v2"}` built the image as `spam-api:multistage-v2b` (a new tag was needed since reusing the same tag string would not have triggered a real `kubectl set image` rollout) loaded it into minikube and updated the Deployment using `kubectl set image`. Checking `kubectl rollout status` and `kubectl rollout history` showed a smooth update with revision 3 active and the old ReplicaSet scaled to 0. Because Kubernetes creates a new pod and waits for its readiness probe before terminating an old one the API maintained availability with zero downtime throughout the rollout. If this was a Job instead of a Deployment Kubernetes would not support rolling updates at all because Job pods run to completion and exit rather than serving continuous traffic.

Deployment vs. Job

A Deployment is designed for long-running always-on services like our web API where Kubernetes maintains desired replica counts replaces failed pods and performs rolling updates without downtime. In contrast a Job is designed for batch workloads like our data validation task that perform a fixed set of completions once and terminate where concepts like readiness probes rolling updates and permanent replica maintenance do not apply.